

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3: Reverse Engineering Task

**COURSE NAME:** Cloud Computing and Big Data Analytics      **COURSE CODE:** CSA15

---

### COURSE OUTCOME COVERED

**CO4:** *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Articulation (Level 4)  
Question Count: 5

**Total Marks: 30**

---

### Code of Conduct

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: \_\_\_\_\_

### Assessment Tasks

#### Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.
2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.
3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.
4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.
5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

### Reverse Engineering Task Rubric

| Criteria                      | Weightage | Level 4 - Exemplary                                  | Level 3 - Proficient      | Level 2 - Developing  | Level 1 - Beginning |
|-------------------------------|-----------|------------------------------------------------------|---------------------------|-----------------------|---------------------|
| Inference Accuracy            | 25%       | Infers system components and flow with high accuracy | Mostly accurate inference | Basic inference       | Weak inference      |
| Understanding of Architecture | 25%       | Shows clear understanding of                         | Good understanding        | Partial understanding | Poor understanding  |

|                           |     |                                              |                     |                  |                        |
|---------------------------|-----|----------------------------------------------|---------------------|------------------|------------------------|
|                           |     | underlying big data architecture             |                     |                  |                        |
| Use of Technical Evidence | 20% | Uses strong reasoning and technical evidence | Reasonable evidence | Limited evidence | No meaningful evidence |
| Depth of Analysis         | 15% | Analysis is deep and insightful              | Reasonably deep     | Basic depth      | Superficial analysis   |
| Clarity                   | 15% | Clear and organized explanation              | Generally clear     | Somewhat unclear | Poorly presented       |

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3: Reverse Engineering Task — Questions with Answers

|                         |                                              |                        |       |
|-------------------------|----------------------------------------------|------------------------|-------|
| <b>COURSE NAME:</b>     | Cloud Computing and Big Data Analytics       | <b>COURSE CODE:</b>    | CSA15 |
| <b>Assessment Tool:</b> | Tool 3 – Reverse Engineering (Weightage 10%) | <b>Total Marks:</b>    | 30    |
| <b>Taxonomy:</b>        | Dave's Taxonomy – Articulation (Level 4)     | <b>Question Count:</b> | 5     |

### COURSE OUTCOME COVERED

**CO4:** Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)

#### Code of Conduct

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: \_\_\_\_\_

**Task Instruction:** For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

### Assessment Tasks — Reverse Engineering (Answers)

#### Question 1

A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.

#### Answer

##### A. Inferred data types

| Signal                         | Data Type                               | Probable Fields                                                         | Inference Evidence                                                             |
|--------------------------------|-----------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Clicks / page views            | Semi-structured JSON event stream       | user_id, session_id, device_id, item_id, page_type, timestamp, referrer | Recommendations react within a session → event-level, not aggregated, capture  |
| Search queries                 | Unstructured text + structured metadata | query_string, filters applied, results shown, clicked rank              | "Searched for X" influencing results implies query text is stored and embedded |
| Purchases                      | Strictly structured transactional       | order_id, user_id, item_id, qty, price, payment status                  | Must be ACID and reconcilable with billing → sourced from an OLTP database     |
| Device / cross-device identity | Structured mapping table                | device_id → user_id, login events, cookie/IDFA graph                    | "Multiple devices" in the prompt → an identity-stitching component must exist  |

|                              |                                                  |                                                     |                                                                    |
|------------------------------|--------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------|
| Item catalogue               | Structured + unstructured (images, descriptions) | category, brand, price, attributes, image vectors   | Content-based recommendations require item features                |
| Implicit & explicit feedback | Structured                                       | dwelt time, add-to-cart, wishlist, returns, ratings | Negative signals (returns) are needed or the model over-recommends |

## B. Inferred ingestion flow

**Client SDK (web/iOS/Android)** → batched HTTPS event collector → **Apache Kafka** (topics: clicks, search, orders; partitioned by user\_id to preserve per-user ordering) → two consumers: **(1) Stream processor** (Flink/Spark Streaming → session state in Redis → real-time features) and **(2) Sink connector** → **Data lake (S3/HDFS, Parquet, partitioned by date/hour)**. Purchases arrive separately via **CDC (Debezium)** from the order database.

- **Why Kafka:** the traffic is bursty and must feed several consumers (real-time model, lake, monitoring) from one write — a database would be a bottleneck and a single point of coupling.
- **Why partition by user\_id:** recommendation logic depends on the ordering of a user's actions; partitioning by user guarantees ordering where it matters without global ordering cost.
- **Why CDC for purchases, events for clicks:** purchases are low-volume and correctness-critical (pull from system of record); clicks are high-volume and tolerant of small loss (push as events).
- **Identity stitching** occurs early — deterministic on login, probabilistic (IP, device fingerprint) for anonymous sessions — otherwise the same person looks like three different users.

## C. Inferred analytics stages

| Stage | Activity                         | Likely Technology                                                  | Design Decision Inferred                                                                                                             |
|-------|----------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Cleansing & sessionisation       | Spark / Flink windows                                              | Bot filtering and 30-minute inactivity session windows — raw clicks are too noisy to model directly                                  |
| 2     | Identity resolution              | Graph / MDM job                                                    | Cross-device unification chosen over per-device models to avoid a cold start on every new device                                     |
| 3     | Feature engineering              | Spark + feature store (Feast)                                      | Same feature definitions used offline and online to prevent training/serving skew                                                    |
| 4     | Candidate generation (retrieval) | ALS / matrix factorisation, item2vec embeddings, ANN index (FAISS) | Two-stage design: narrow millions of items to ~500 candidates cheaply — ranking every item per request is computationally impossible |
| 5     | Ranking                          | Gradient boosting / deep ranking model                             | An expensive model is affordable only on the small candidate set; optimises CTR/conversion                                           |
| 6     | Business re-ranking              | Rules layer                                                        | Injects margin, stock availability, diversity and freshness — pure relevance would recommend out-of-stock or repetitive items        |
| 7     | Real-time serving                | Low-latency API + Redis/Cassandra cache                            | Pre-computed recommendations cached; only the last few events applied live to hit a <100 ms budget                                   |
| 8     | Evaluation & feedback            | A/B testing, offline metrics (recall@k, NDCG)                      | Closed loop — outcomes are logged back as training data, so the system continuously self-improves                                    |

**Key design decisions inferred:** (i) a **two-stage retrieve-then-rank** architecture for latency/quality balance; (ii) a **Lambda-style split** — batch model training nightly, real-time signals applied at request time; (iii) **hybrid recommendation** (collaborative + content-based) to survive cold start for new users and new items; (iv) **cross-device identity as a first-class component**, without which personalisation quality collapses.

## Question 2

A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.

### Answer

#### A. What the visible features reveal

- **Live heatmap** → continuous GPS ingestion, geospatial aggregation and sub-minute refresh → a **stream processing** backbone must exist.
- **Demand forecast** → historical time-series plus ML → a **batch/training** backbone must also exist → therefore a **Lambda (or Kappa) hybrid architecture**.
- **Surge pricing / ETA** → the output is fed back into the app in real time → a low-latency serving store sits in front of the analytics layer.

#### B. Inferred component stack

| Layer                      | Probable Technology                                                         | Reason for the Inference                                                                                                                        |
|----------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingestion                  | Apache Kafka / Amazon Kinesis; MQTT or gRPC from driver apps                | Millions of GPS pings every few seconds from moving vehicles; needs buffering, partitioning by city/geohash and replay                          |
| Stream processing          | Apache Flink or Spark Structured Streaming; tumbling/sliding windows        | Heatmap tiles must recompute every 10–60 s; Flink's event-time and watermark handling copes with late GPS pings from poor network areas         |
| Geospatial indexing        | H3 hexagons / geohash / S2 cells; PostGIS for static zones                  | Heatmaps are drawn on grid cells, not raw coordinates — hexagonal binning (Uber's H3) is the standard choice because neighbours are equidistant |
| Hot / serving store        | Redis, Cassandra or ScyllaDB keyed by cell + time bucket                    | Map tiles are read enormously more often than written; needs single-digit ms reads and TTL expiry                                               |
| Distributed storage (cold) | HDFS / Amazon S3 data lake, Parquet/ORC, partitioned by city/date/hour      | Years of trip history retained for forecasting, pricing analysis and regulatory reporting; columnar format for cheap analytical scans           |
| Time-series store          | InfluxDB / TimescaleDB / Druid                                              | Dashboards query "rides per zone per minute" — Druid/OLAP stores answer such rollups in sub-second                                              |
| Batch processing           | Apache Spark on YARN/Kubernetes, orchestrated by Airflow                    | Nightly aggregation, feature building and model retraining over petabyte history                                                                |
| Forecasting models         | Gradient boosting / LSTM / Prophet with weather, event and holiday features | Demand is strongly seasonal and weather-dependent; a purely historical average would fail on anomalies                                          |
| Transactional store        | Sharded RDBMS or DynamoDB for bookings and payments                         | Trip and payment records need ACID guarantees that an analytics store does not provide — hence a separate OLTP path                             |
| Serving APIs               | Microservices behind a gateway; WebSocket push to apps                      | Map updates arrive without the app polling — implies a push channel                                                                             |

### C. Inferred data flow

**Driver & rider apps** (GPS every 4–5 s, ride requests) → **Kafka** (partitioned by city/geohash) → **Speed layer**: Flink → map-match, snap to H3 cell, window-aggregate → **Redis/Cassandra** → heatmap & surge API → app.  
→ **Batch layer**: Kafka sink → **S3/HDFS lake** → Spark (Airflow-scheduled) → feature store → **demand forecast model** → predictions written back to the serving store → driver-positioning recommendations.

### D. Design decisions likely made and why

| Decision                                              | Alternative Rejected                 | Probable Justification                                                                                                    |
|-------------------------------------------------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Hexagonal cell aggregation (H3)                       | Raw lat/long queries or square tiles | Fixed cells make aggregation a simple group-by; hexagons avoid the distorted corner distances of squares                  |
| Lambda architecture (speed + batch)                   | Batch only, or streaming only        | Live map needs seconds; forecasting needs long, accurate, re-computable history — one layer cannot serve both             |
| Partition by city / region                            | Global single partition set          | Ride-sharing is inherently a per-city market; it isolates load, enables regional scaling and satisfies data-residency law |
| Eventual consistency in the heatmap                   | Strong consistency                   | A 20-second-stale heatmap is acceptable; strong consistency at this write rate would be prohibitively expensive           |
| Separate OLTP store for trips/payments                | Single store for everything          | Money requires ACID; analytics requires scan throughput — conflicting optimisations                                       |
| Approximate algorithms (HLL, sampling) for dashboards | Exact counts                         | Cardinality estimates are adequate for operational monitoring at a fraction of the compute                                |
| Retention tiering of raw GPS                          | Keep all raw pings forever           | Raw traces are enormous and privacy-sensitive; aggregated cell data retained long-term, raw traces expired                |

**5V reading:** Velocity dominates (continuous GPS with a hard freshness SLA), Volume is very high but compressible by aggregation, Variety is moderate (mostly structured telemetry plus weather/event enrichment), Veracity is a real problem (GPS drift in urban canyons and tunnels → map-matching and outlier filtering are mandatory), and Value appears as better ETAs, higher driver utilisation and surge revenue.

#### Question 3

A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.

#### Answer

##### A. Inferred streaming mechanism

| Component      | Probable Technology                                                | Evidence / Reasoning                                                                                                                                      |
|----------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Event backbone | Apache Kafka (or Kinesis), topics partitioned by card/account hash | "Near real time" over card volumes implies a durable, replayable, high-throughput bus; partitioning by card keeps a card's events ordered on one consumer |

|                       |                                                                                                           |                                                                                                                                               |
|-----------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Stream engine         | Apache Flink (or Spark Structured Streaming)                                                              | Needs stateful, per-card windowed features ("3 countries in 1 hour"), exactly-once semantics and event-time handling — Flink's core strengths |
| State & feature cache | RocksDB-backed Flink state; Redis for shared profile lookups                                              | Velocity features must be read in single-digit milliseconds inside the authorisation window                                                   |
| Detection logic       | Hybrid: deterministic rule engine (Drools) + ML scorer (XGBoost / autoencoder / isolation forest)         | Rules catch known patterns and give explainability; ML catches novel patterns — production systems almost always run both                     |
| Graph analysis        | Neo4j / GraphX for merchant–card–device linkage                                                           | Fraud rings are detected through shared devices and merchants, not through single transactions                                                |
| Storage               | Encrypted S3/HDFS lake (history & retraining) + Cassandra (recent profiles) + Elasticsearch (case search) | Three different access patterns → polyglot persistence                                                                                        |
| Dashboard             | Kibana / Grafana / custom case-management UI over Elasticsearch                                           | "Dashboard that flags" implies an investigator queue with search and drill-down, not just a chart                                             |

## B. Inferred processing flow (latency budget ~100–300 ms)

**POS / online payment** → TLS → **Tokenisation at the edge (PAN → token)** → **Kafka** → **Flink job**: enrich (customer profile, merchant risk, geolocation) → compute velocity features (txn count/amount in 1 min, 1 h, 24 h; distance from last txn; device/IP novelty) → **rules + ML score** → decision {approve | step-up authentication | decline | flag} → **alert topic** → case-management queue + customer SMS/push → analyst feedback → **lake** → nightly retraining.

## C. Inferred security mechanisms

| Security Control         | Likely Implementation                                                                 | Why It Must Be Present                                                      |
|--------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Tokenisation / PAN vault | Card number replaced at the edge; vault isolated in a separate trust zone             | PCI-DSS — keeps the analytics cluster out of full cardholder-data scope     |
| Encryption in transit    | TLS 1.3 externally, mTLS between Kafka brokers and clients                            | Transaction streams crossing many hops must be unreadable if intercepted    |
| Encryption at rest       | HDFS/S3 encryption zones, HSM/KMS-managed keys with rotation                          | Protects the historical lake, backups and snapshots                         |
| Access control           | Kerberos/OIDC + MFA; Ranger RBAC with column- and row-level masking                   | Analysts see masked PAN (first6/last4) and only their assigned case segment |
| Topic-level ACLs         | Kafka ACLs per producer/consumer service account                                      | A compromised consumer must not be able to read or publish to other topics  |
| Immutable audit trail    | WORM logs of every access and decision, streamed to a SIEM                            | Regulatory evidence; also protects the bank in customer disputes            |
| Network isolation        | Private subnets, no public endpoints, segmentation between edge, processing and vault | Limits lateral movement after any single compromise                         |
| Model & data integrity   | Signed model artefacts, versioned registry, adversarial/drift monitoring              | Fraudsters actively probe and attempt to poison detection models            |
| Privacy compliance       | Data minimisation, retention limits, region-pinned storage                            | GDPR/DPDP and local data-localisation rules                                 |

## D. Inferred alerting mechanism

- **Tiered response, not a single threshold:** low score → approve silently; medium → step-up authentication (OTP/3-D Secure); high → block and alert; very high → auto-freeze the card. This tiering is inferred because a binary block/allow rule would generate intolerable false declines.
- **Channels:** customer SMS/push/e-mail, investigator queue in the case-management UI, PagerDuty/Slack for platform anomalies (e.g. sudden spike in decline rate).
- **Alert suppression and deduplication:** grouping by card and by fraud ring, plus rate limiting — without this, one compromised BIN range would flood the queue.
- **Prioritisation:** alerts ranked by exposure amount × probability so limited analyst time targets the largest expected loss.
- **Feedback loop:** the analyst's confirm/reject verdict becomes a labelled training row — this is the mechanism by which the model keeps up with evolving fraud.

**Central design decision inferred:** the system is tuned on the **false positive vs. false negative trade-off**. A missed fraud costs the transaction value; a false decline costs customer trust and future revenue. The tiered action model, the rules+ML hybrid and the human-in-the-loop review queue all exist to manage precisely that trade-off, rather than to maximise raw detection accuracy.

### Question 4

A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.

### Answer

#### A. Classification of the platform's data

| Data                                                  | Type                                       | Probable Storage                                                                 | Reason for that Choice                                                                                         |
|-------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Videos & images                                       | Unstructured, large binary                 | Object storage (S3/HDFS/Blob) + CDN edge caching; multiple transcoded renditions | Databases cannot economically store or serve GB-scale binaries; CDN is required for global playback latency    |
| Text comments & captions                              | Unstructured text with structured metadata | Wide-column store (Cassandra/HBase) keyed by post_id; mirrored to Elasticsearch  | Massive write rate, append-heavy, always read by parent post → a natural partition key                         |
| Hashtags                                              | Semi-structured tag arrays                 | Inverted index (Elasticsearch) + Redis sorted sets for trending counters         | Queried as "all posts with #tag", plus real-time top-N trending — both index problems, not table scans         |
| Engagement metrics (likes, views, shares, watch time) | Structured counters & time-series          | Redis/HBase counters → rolled up into Druid/ClickHouse; raw events to the lake   | Extremely high write rate on a small value — counter-optimised stores, not row updates in an RDBMS             |
| User profiles & social graph                          | Structured / graph                         | Sharded RDBMS or key-value store; graph store (or adjacency lists) for follows   | Feed generation is a graph traversal problem                                                                   |
| Derived AI metadata                                   | Structured, machine-generated              | Feature store + lake tables                                                      | Video labels, embeddings, moderation scores and transcripts turn unstructured content into queryable structure |

#### B. The unifying pattern — metadata + pointer



**The binary lives in object storage; the database stores only a pointer plus rich metadata.**

Example post record: {*post\_id*, *user\_id*, *created\_at*, *video\_uri*: "s3://.../v\_9f2.mp4", *duration*, *resolution*, *hashtags*[], *caption*, *language*, *ai\_labels*[], *embedding\_id*, *moderation\_score*, *like\_count*}

- This is the key inference: **structured and unstructured data are joined by identifiers, not co-located**. A feed query touches only small metadata rows; the heavy video is fetched separately by the CDN.
- ML pipelines convert unstructured content into structured vectors and labels — after that, "unstructured" video becomes searchable and recommendable through ordinary structured operations.

### C. Inferred pipeline

**Upload** → object store (raw) → **transcoding farm** (multiple bitrates/thumbnails) → **AI enrichment** (object detection, ASR transcript, NSFW/moderation, embeddings) → **metadata written to Cassandra + Elasticsearch + feature store**. In parallel: **engagement events** → Kafka → Flink (real-time counters, trending hashtags) → Redis/Druid; and Kafka → **lake (Parquet)** → Spark batch → recommendation and ranking models → back into the feed service.

### D. Design decisions likely made

| Decision                                                           | Trade-off Accepted                                 | Justification                                                                                                            |
|--------------------------------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Polyglot persistence</b> — a different store per access pattern | Higher operational complexity; more systems to run | No single database can serve blobs, counters, full-text search and graph traversal well                                  |
| <b>Data lake + lakehouse</b> as the single analytical copy         | Query latency higher than a warehouse              | Lets one platform hold raw video, JSON events and curated tables together with schema-on-read                            |
| <b>Denormalisation and pre-computed feeds</b>                      | Storage duplication, write amplification           | Reads outnumber writes by orders of magnitude; the feed must render in milliseconds                                      |
| <b>Eventual consistency on counters</b>                            | Like count may lag a few seconds                   | Strong consistency on a viral post would be a severe write hotspot; slight lag is imperceptible to users                 |
| <b>Approximate counting (HyperLogLog) for unique views</b>         | Small error margin                                 | Exact distinct counts across billions of events are not worth the compute                                                |
| <b>ML-generated structure over unstructured content</b>            | Model cost and error rate                          | The only practical way to search, moderate and recommend video at scale                                                  |
| <b>Tiered storage &amp; CDN</b>                                    | Complex lifecycle management                       | A small fraction of content receives most views; cold content moves to cheap archive tiers                               |
| <b>Privacy &amp; governance layer</b>                              | Extra engineering effort                           | Right-to-erasure, regional data residency and content moderation obligations apply to all copies, including derived ones |

**Conclusion:** The platform does not "store everything in one system". It **separates the payload from its description** — unstructured payloads in cheap, scalable object storage, structured descriptions in fast, queryable stores — and uses machine learning as the bridge that continuously converts the former into the latter.

### Question 5

A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

### Answer

#### A. Inferred data sources

| Source                                                                 | Type                   | Rate                           | Analytical Purpose                                  |
|------------------------------------------------------------------------|------------------------|--------------------------------|-----------------------------------------------------|
| PLC / SCADA sensor tags<br>(vibration, temperature, pressure, current) | Structured time-series | Very high — ms to sec sampling | Anomaly detection, predictive maintenance           |
| Machine / controller logs                                              | Semi-structured text   | High, bursty                   | Fault-code analysis, root-cause tracing             |
| Maintenance events<br>(CMMS/SAP PM work orders)                        | Structured, low volume | Days apart                     | The <b>labels</b> for supervised failure prediction |
| Quality inspection & vision images                                     | Unstructured           | Per batch/part                 | Defect correlation with machine condition           |
| Production/MES and ERP data                                            | Structured             | Batch                          | OEE calculation, cost of downtime                   |

#### B. Inferred integration tools

| Tool / Standard                                                                 | Role                                                                  | Why It Is Almost Certainly Present                                                                      |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>OPC-UA gateway</b>                                                           | Reads PLC/SCADA tags from shop-floor equipment                        | The de facto industrial protocol standard; legacy machines speak Modbus/OPC, not HTTP                   |
| <b>MQTT broker</b> (Mosquitto, HiveMQ)                                          | Lightweight publish/subscribe transport from plant to platform        | Low overhead and tolerant of intermittent plant connectivity                                            |
| <b>Edge gateway / fog node</b><br>(AWS Greengrass, Azure IoT Edge, NiFi MiNiFi) | Local buffering, filtering, downsampling, first-line inference        | "Many plants" → WAN links are limited and sometimes drop; raw full-rate telemetry cannot all be shipped |
| <b>Apache NiFi</b>                                                              | Protocol translation, routing, provenance across heterogeneous plants | Different plants have different vendors and formats — NiFi normalises them                              |
| <b>Apache Kafka</b>                                                             | Central multi-tenant event backbone, topic per plant/asset class      | Decouples plants from consumers; buffers and replays                                                    |
| <b>Sqoop / JDBC / REST connectors</b>                                           | Pull maintenance work orders and ERP master data                      | Failure labels live in SAP PM or a CMMS, not in the sensor stream                                       |
| <b>Apache Airflow</b>                                                           | Orchestrates daily aggregation and model retraining                   | Multi-stage dependent pipelines with retries and SLAs                                                   |
| <b>Schema registry</b>                                                          | Manages evolving sensor tag schemas across plants                     | Adding a new machine type must not break existing consumers                                             |

#### C. Inferred storage strategy

- **Tiered / medallion lake:** *Bronze* raw telemetry → *Silver* cleaned, unit-standardised, gap-filled → *Gold* per-asset feature tables and OEE marts.
- **Hot tier:** a time-series database (InfluxDB, TimescaleDB) or Druid holding roughly the last 30–90 days at full resolution for operator dashboards and alerts.
- **Cold tier:** HDFS/S3 in Parquet or ORC, partitioned by *plant\_id / asset\_id / date*, compressed — multi-year history is required because failures are rare events.

- **Downsampling & roll-ups:** raw millisecond data kept only briefly; 1-second, 1-minute and hourly aggregates retained long-term — the standard IoT cost control.
- **Small-file compaction:** thousands of tiny device files are compacted into large Parquet files, otherwise the HDFS NameNode metadata load becomes the bottleneck.
- **Unified asset model / digital twin registry:** a hierarchy of plant → line → machine → sensor tag so that identical machines in different plants are comparable — without it cross-plant benchmarking is impossible.
- **Storage of images/vision data** in object storage with pointers held in the metadata catalogue.

#### D. Inferred analytics path

**Edge inference** (immediate threshold and vibration alarms, ms latency) → **Stream analytics** (Flink/Spark: rolling windows, drift and anomaly detection, minutes) → **Batch analytics** (Spark: feature engineering — RMS vibration, kurtosis, FFT bands, duty cycles, temperature gradients) → **ML** (anomaly detection on unlabelled data; survival / Remaining-Useful-Life regression and classification where maintenance labels exist) → **Prescriptive** (maintenance scheduling optimisation) → **Consumption** (SCADA/Grafana dashboards, CMMS work-order auto-creation, OEE reports).

| Analytics Stage | Question Answered                   | Technique Inferred                                                           |
|-----------------|-------------------------------------|------------------------------------------------------------------------------|
| Descriptive     | What is machine uptime and OEE?     | Aggregation, dashboards, control charts                                      |
| Diagnostic      | Why did line 3 stop last night?     | Fault-code correlation, log tracing, Pareto analysis                         |
| Predictive      | Which bearing will fail and when?   | Vibration spectral features + RUL model / survival analysis                  |
| Prescriptive    | When should we schedule the repair? | Optimisation balancing downtime cost, spare availability and production plan |

#### E. Key design decisions and their justification

| Decision                                       | Alternative Rejected                                 | Justification                                                                                                           |
|------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Edge pre-processing before transmission        | Ship all raw telemetry to the cloud                  | Plant WAN bandwidth and cost; safety-critical alarms cannot depend on a cloud round trip                                |
| Time-series DB for hot data, lake for cold     | One store for everything                             | Operators need sub-second dashboards; data scientists need multi-year scans — opposite optimisations                    |
| Downsampling with retained aggregates          | Retain everything at full resolution                 | Full-rate multi-year retention across many plants is economically indefensible                                          |
| Joining sensor data to CMMS work orders        | Unsupervised anomaly detection alone                 | Without maintenance labels, alerts cannot be tied to actual failure modes — this join is what makes prediction possible |
| Unified asset hierarchy across plants          | Per-plant independent models                         | Transfer learning across identical machines fixes the sparse-failure-data problem                                       |
| OT/IT network segmentation (DMZ, one-way flow) | Direct connection of shop floor to corporate network | Compromise of a control system is a physical-safety event, not merely a data breach                                     |

**5V reading:** Volume and Velocity are driven by sensor sampling rates, Variety is moderate but the **protocol** variety is high (OPC-UA, Modbus, MQTT, proprietary), Veracity is the hardest problem (sensor drift, calibration errors, missing readings during network outages), and Value is measured directly as **reduced unplanned downtime and extended asset life** — the clearest ROI case in all of industrial big data.

— *End of Assessment Tool 3 (Reverse Engineering Task)* —